Problem Statement:

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array. The *Player* class is provided in the editor below. It has two fields:

1. : a string.
2. : an integer.

Given an array of *Player* objects, write a comparator that sorts them in order of decreasing score. If or more players have the same score, sort those players alphabetically ascending by name. To do this, you must create a *Checker* class that implements the *Comparator* interface, then write an *int compare(Player a, Player b)* method implementing the Comparator.compare(T o1, T o2) method. In short, when sorting in ascending order, a comparator function returns if , if , and if .

Declare a *Checker* class that implements the *comparator* method as described. It should sort first descending by score, then ascending by name. The code stub reads the input, creates a list of Player objects, uses your method to sort the data, and prints it out properly.

Example

Sort the list as . Sort first descending by score, then ascending by name.

Input Format

The first line contains an integer, , the number of players.
Each of the next lines contains a player's and , a string and an integer.

Constraints

-
- Two or more players can have the same name.
- Player names consist of lowercase English alphabetic letters.

Output Format

You are not responsible for printing any output to stdout. Locked stub code in *Solution* will instantiate a *Checker* object, use it to sort the *Player* array, and print each sorted element.

Program:

```
import java.util.*;


// Player class with name and score
class Player {
   String name;
   int score;
```

```
    Player(String name, int score) {
        this.name = name;
        this.score = score;
    }
}

// Checker class implementing Comparator
class Checker implements Comparator<Player> {
    @Override
    public int compare(Player a, Player b) {
        // First compare by score (descending)
        if (a.score != b.score) {
            return b.score - a.score; // higher score comes first
        } else {
            // If scores are equal, compare by name (ascending)
            return a.name.compareTo(b.name);
        }
    }
}

public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(); // number of players
        Player[] players = new Player[n];

        for (int i = 0; i < n; i++) {
            String name = sc.next();
            int score = sc.nextInt();
            players[i] = new Player(name, score);
```

```
        }

        // Sort using Checker comparator
        Arrays.sort(players, new Checker());

        // Print sorted players
        for (Player p : players) {
            System.out.println(p.name + " " + p.score);
        }

        sc.close();
    }
}
```

Sample Input

5

Amy 100

David 100

Heraldo 50

Aakansha 75

Aleksa 150

Sample Output

Aleksa 150

Amy 100

David 100

Aakansha 75

Heraldo 50